Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНЖЕНЕРНЫЙ БИЗНЕС И МЕНЕДЖМЕНТ»
КАФЕДРА «ПРОМЫШЛЕННАЯ ЛОГИСТИКА»

ОТЧЕТ
Предмет: Разработка интернет приложений
Лабораторные работы №3
“Разработка клиентской части веб-приложения”

Студент: Сигорский Александр Евгеньевич
Группа: ИБМ3-54Б
Преподаватель: М.В. Бурмистрова

Москва 2026

1. Описание задания

Цель лабораторной работы — рассмотреть разработку клиентской части веб-приложения с использованием HTML, CSS и JavaScript: сформировать структуру страницы, организовать навигацию, обработку пользовательских действий и динамическое изменение содержимого без полной перезагрузки браузера.

В качестве практического проекта используется CodeSlicer. Это local-first система анализа программного кода, которая строит карту зависимостей проекта, помогает оценивать влияние изменений и показывает результаты через браузерный интерфейс. Основная идея CodeSlicer состоит в том, что программный проект рассматривается как связанная система: функции, классы, API, сервисы, данные и тесты образуют граф.

Визуальным ориентиром при оформлении лабораторного проекта служила Kodik IDE: от неё был взят общий образ современного минималистичного интерфейса для разработчика. При этом CodeSlicer является отдельным приложением и выполняет самостоятельную задачу — формирует проверяемый контекст о структуре проекта и последствиях изменений.

2. Место лабораторной работы в CodeSlicer

Клиентская часть находится в каталоге frontend. Основными файлами являются:
• frontend/index.html — структура интерфейса и основные экраны;
• frontend/styles.css и frontend/simple.css — визуальное оформление;
• frontend/app.js — состояние приложения, маршрутизация, DOM, обработчики событий и визуализация;
• frontend/api-client.js — клиент для обращения к локальному HTTP API.

Пользователь работает с четырьмя основными режимами: «Проверка изменений», «Карта проекта», «Локальные источники» и «Graphify». Требования лабораторной работы выполняются внутри реального одностраничного приложения, а не на отдельном учебном макете.

3. Текст программы и пояснения

3.1. HTML-структура верхней панели

Основная HTML-страница задаёт навигацию и форму выбора проекта.

```
<header class="simple-topbar">
  <button class="brand" type="button" data-route="review"
          aria-label="Открыть проверку изменений">
    <span class="cs-logo" aria-hidden="true">
      <i></i><i></i><i></i><i></i>
    </span>
    <span>CodeSlicer</span>
  </button>

  <nav class="simple-nav" aria-label="Основная навигация">
    <a href="#review" data-route="review">Проверка изменений</a>
    <a href="#map" data-route="map">Карта проекта</a>
    <a href="#sources" data-route="sources">Локальные источники</a>
    <a href="#graphify" data-route="graphify">Graphify</a>
  </nav>

  <div class="simple-project-picker">
    <input id="projectPath" type="text"
           placeholder="Путь к локальному проекту">
    <button class="button primary" id="analyzeButton"
            type="button">Построить карту</button>
  </div>
</header>
```

Листинг 1 — верхняя панель и навигация CodeSlicer.

Атрибуты data-route связывают HTML-разметку с JavaScript-маршрутизацией. При нажатии на пункт меню браузер не загружает новую HTML-страницу. JavaScript определяет выбранный маршрут и

переключает нужный раздел интерфейса. Это позволяет реализовать поведение Single Page Application без отдельного frontend-фреймворка.

3.2. Экран первоначального анализа

Если граф ещё не построен, приложение показывает onboarding-форму. Пользователь вводит путь к локальному проекту и запускает анализ.

```
<section id="onboarding" class="simple-onboarding" hidden>
  <span class="eyebrow">НАЧАТЬ</span>
  <h1>Постройте карту проекта</h1>
  <p>Укажите папку с кодом. CodeSlicer построит локальный граф связей
— код никуда не отправляется.</p>
  <input id="onboardingPath" type="text"
         placeholder="C:\Projects\my-app">
  <button class="button primary" id="onboardingAnalyze"
          type="button">Построить карту</button>
  <div id="progressPanel" class="progress-panel" hidden>
    <strong id="progressStage">Подготовка</strong>
    <span id="progressPercent">0%</span>
    <p id="progressMessage">Готовлю анализ…</p>
  </div>
</section>
```

Листинг 2 — форма запуска анализа.

Блоки уже присутствуют в DOM, но JavaScript управляет их видимостью. По мере выполнения анализа изменяются процент, стадия и текст сообщения. В результате пользователь получает обратную связь без перезагрузки страницы.

3.3. Объект состояния интерфейса

В frontend/app.js используется единый объект state. В нём сохраняются данные, необходимые разным экранам приложения.

```
const state = {
  ready: false,
  project: '',
  hasAnalysis: false,
  analysis: null,
  graph: null,
  overview: null,
  inventory: null,
  adapters: [],
  tools: [],
  review: null,
  projection: null,
  selectedEntity: '',
  selectedFile: '',
  route: 'review',
  pending: new Set(),
};
```

Листинг 3 — часть состояния приложения.

Централизованное состояние важно, потому что одни и те же данные используются несколькими представлениями. Путь к проекту нужен как для карты, так и для review; информация о графе влияет на доступность нескольких экранов; выбранная сущность используется в инспекторе и цепочке влияния.

3.4. Навигация между экранами

Маршрут хранится в hash-части адреса. Функция navigate изменяет location.hash, после чего renderRoute выбирает нужный экран.

```
function navigate(route, params = {}) {
  const canonical = normalizeRoute(route);
```

```
  const query = new URLSearchParams(params).toString();
  const hash = `#${canonical}${query ? `?${query}` : ''}`;

  if (location.hash !== hash) {
    location.hash = hash;
  } else {
    renderRoute();
  }
}

function renderRoute() {
  state.route = normalizeRoute(location.hash);

  $$('.route-view').forEach((view) =>
    view.classList.toggle(
      'active',
      view.dataset.routeView === state.route
    )
  );

  if (!state.ready || !state.hasAnalysis) {
    showOnboarding(true);
    return;
  }

  showOnboarding(false);

  if (state.route === 'review') loadReview();
  if (state.route === 'map') loadProjection();
  if (state.route === 'sources') loadSources();
  if (state.route === 'graphify') loadGraphify();
}
```

Листинг 4 — маршрутизация внутри одностраничного приложения.

Переход по интерфейсу состоит из двух действий: переключение CSS-состояния и загрузка требуемых данных. Общая оболочка страницы не перезагружается. Это уменьшает количество полных загрузок документа и делает навигацию быстрее.

3.5. Создание динамических DOM-элементов

Для динамических карточек и сообщений используются вспомогательные функции.

```
function el(tag, className, label) {
  const node = document.createElement(tag);

  if (className) node.className = className;
  if (label !== undefined) node.textContent = label;

  return node;
}

function append(parent, ...children) {
  children
    .filter(Boolean)
    .forEach((child) => parent.append(child));

  return parent;
}

function stateCard(title, detail, kind = '') {
  const node = el('div', `state-card ${kind}`.trim());

  append(
    node,
    el('strong', '', title),
    detail ? el('p', '', detail) : null
  );
```

```
  return node;
}
```

Листинг 5 — создание элементов интерфейса средствами DOM API.

Основные динамические значения вставляются через textContent. Полученная строка отображается как текст, а не как произвольный HTML. Это важно для интерфейса анализа кода: названия файлов, сущностей и сообщения backend не должны случайно интерпретироваться браузером как HTML-разметка.

3.6. Делегирование событий

Интерфейс содержит много динамических кнопок. Вместо назначения отдельного listener каждой карточке используется общий обработчик click.

```
document.addEventListener('click', (event) => {
  const route = event.target.closest?.('[data-route]');

  if (route) {
    event.preventDefault();
    closeOverlays();
    navigate(route.dataset.route);
    return;
  }

  const action = event.target.closest?.('[data-action]');

  if (action) {
    const name = action.dataset.action;

    if (name === 'refresh-review') {
      state.review = null;
      loadReview(true);
    }
```

```
    if (name === 'analyze') {
      startAnalysis($('#projectPath').value);
    }

    return;
  }
});

listen('#analyzeButton', 'click',
  () => startAnalysis($('#projectPath').value));

window.addEventListener('hashchange', renderRoute);
```

Листинг 6 — обработка пользовательских действий.

С помощью closest определяется ближайший элемент с нужным data-атрибутом. Поэтому обработчик работает и с элементами, которые были созданы после загрузки страницы. Такой подход называется делегированием событий и особенно удобен для интерфейсов, которые активно перестраивают содержимое DOM.

3.7. Интерактивная SVG-карта

Карта проекта не является статической картинкой. JavaScript управляет viewBox SVG, реализуя масштабирование и панорамирование.

```
svg.addEventListener('wheel', (event) => {
  event.preventDefault();

  const rect = svg.getBoundingClientRect();

  const originX =
    view.x +
    (event.clientX - rect.left) /
```

```
    Math.max(1, rect.width) *
    view.width;

  const originY =
    view.y +
    (event.clientY - rect.top) /
    Math.max(1, rect.height) *
    view.height;

  zoom(
    event.deltaY < 0 ? 1.16 : 1 / 1.16,
    originX,
    originY
  );
}, { passive: false });

svg.addEventListener('keydown', (event) => {
  if (event.key === '+' || event.key === '=') {
    zoom(1.16);
  } else if (event.key === '-') {
    zoom(1 / 1.16);
  } else if (event.key === '0') {
    reset();
  }
});
```

Листинг 7 — управление интерактивной картой.

Масштаб вычисляется относительно положения курсора, поэтому пользователь сохраняет визуальный фокус на интересующей области. Помимо колеса мыши, предусмотрено управление клавиатурой, что делает компонент удобнее для разных способов взаимодействия.

3.8. Загрузка проекции графа

При переходе на карту выполняется запрос к API. Полученные данные сохраняются в state.projection и затем передаются функции визуализации.

```
async function loadProjection(force = false) {
  if (!state.project) return;

  const filters = {
    node_kinds: [],
    edge_kinds: [],
    evidence_sources: [],
    min_confidence: 0
  };

  const response =
    await ImpactApi.graphProjection({
      project_path: state.project,
      level: state.mapLevel,
      query: '',
      filters,
      show_all: state.mapScope === 'all',
      max_nodes: 120,
      max_edges: 200
    });

  state.projection = response;

  renderProjection(response);
  updateEntitySuggestions(response.nodes || []);
}
```

Листинг 8 — асинхронная загрузка данных карты.

Ограничения max_nodes и max_edges позволяют интерфейсу сохранять отзывчивость даже на больших проектах. Для обзорного режима нет

необходимости сразу отрисовывать весь граф, если пользователю требуется только общая структура.

3.9. Начальная инициализация

После загрузки страницы bootstrap проверяет local API и получает состояние проекта.

```
async function bootstrap() {
  try {
    const [health, current] =
      await Promise.all([
        ImpactApi.health(),
        ImpactApi.state()
      ]);

    if (health.status !== 'ok') {
      throw new Error('local API не готов');
    }

    const configuredProject =
      current.project_path || '';

    const projectAvailable =
      current.project_exists !== false;

    state.project =
      projectAvailable ? configuredProject : '';

    state.hasAnalysis =
      Boolean(
        current.has_analysis &&
        projectAvailable
      );

    state.analysis = current.analysis || null;
```

```
    state.ready = true;

    if (state.hasAnalysis) {
      await hydrateAnalysis();
      renderRoute();
      showState('Карта проекта готова.');
    } else {
      renderRoute();
    }
  } catch (error) {
    state.ready = true;
    renderRoute();
    showState(
      `Локальный API недоступен: ${error.message}`,
      'error-state'
    );
  }
}

bootstrap();
```

Листинг 9 — начальная загрузка интерфейса.

Promise.all запускает два независимых запроса одновременно. Если сервер недоступен, приложение не создаёт фиктивный успешный результат: пользователь получает сообщение о недоступности API.

4. Экранные формы с примерами выполнения программы

В данном разделе используются реальные изображения CodeSlicer из опубликованных материалов проекта. Они показывают фактический интерфейс и результаты работы системы.

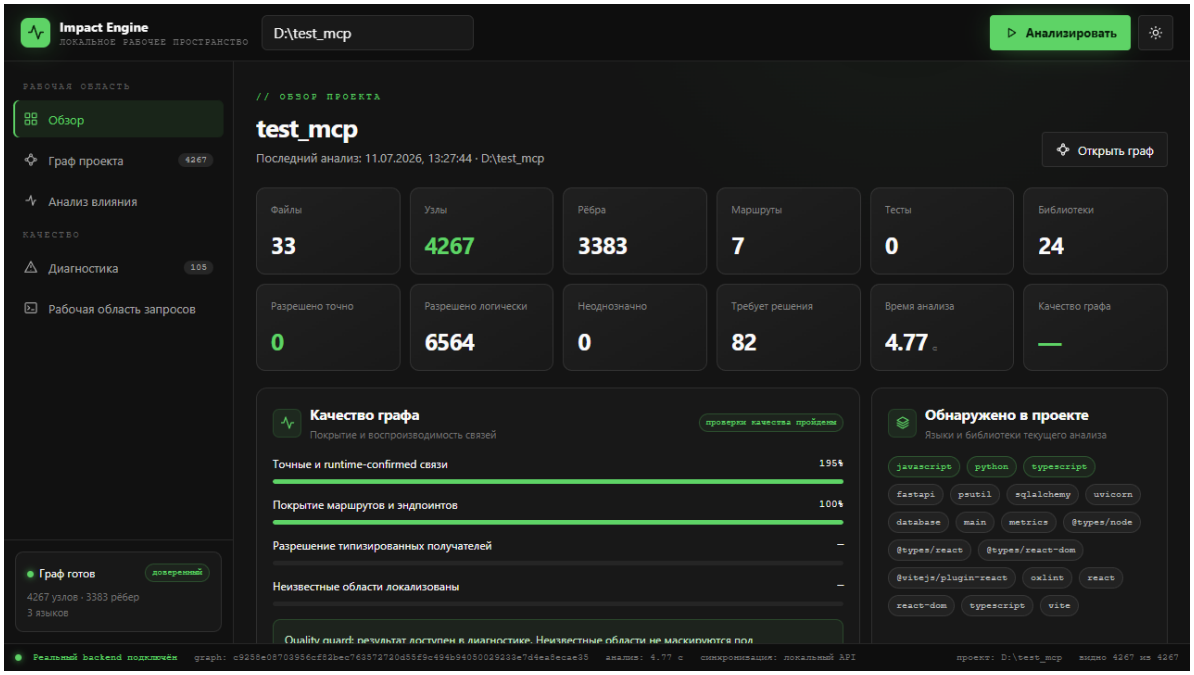


Рисунок 1 — обзор локального интерфейса CodeSlicer.

Экран демонстрирует рабочую область, в которой пользователь получает результаты анализа и переходит между режимами. Форма связана с рассмотренными выше HTML-секциями, hash-навигацией и объектом state.

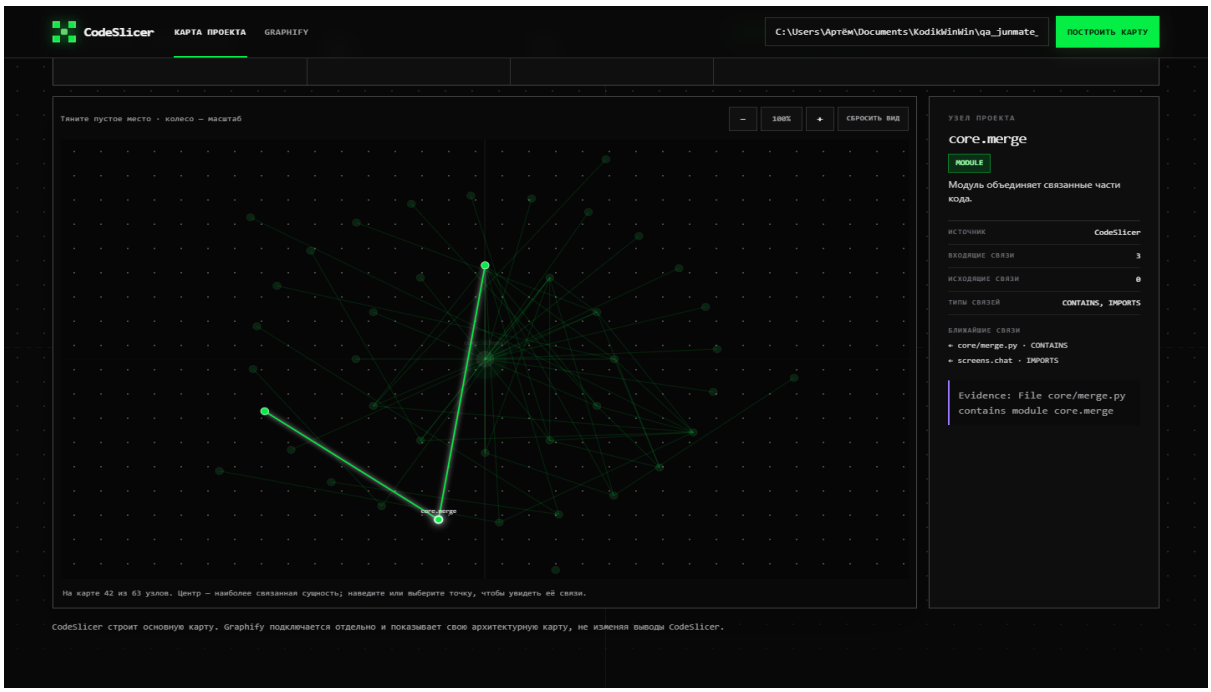


Рисунок 2 — интерактивная карта проекта CodeSlicer.

На изображении видна визуализация узлов и связей. Это результат работы функций загрузки проекции и отрисовки интерактивной карты. Пользователь может масштабировать представление и исследовать отдельные сущности.

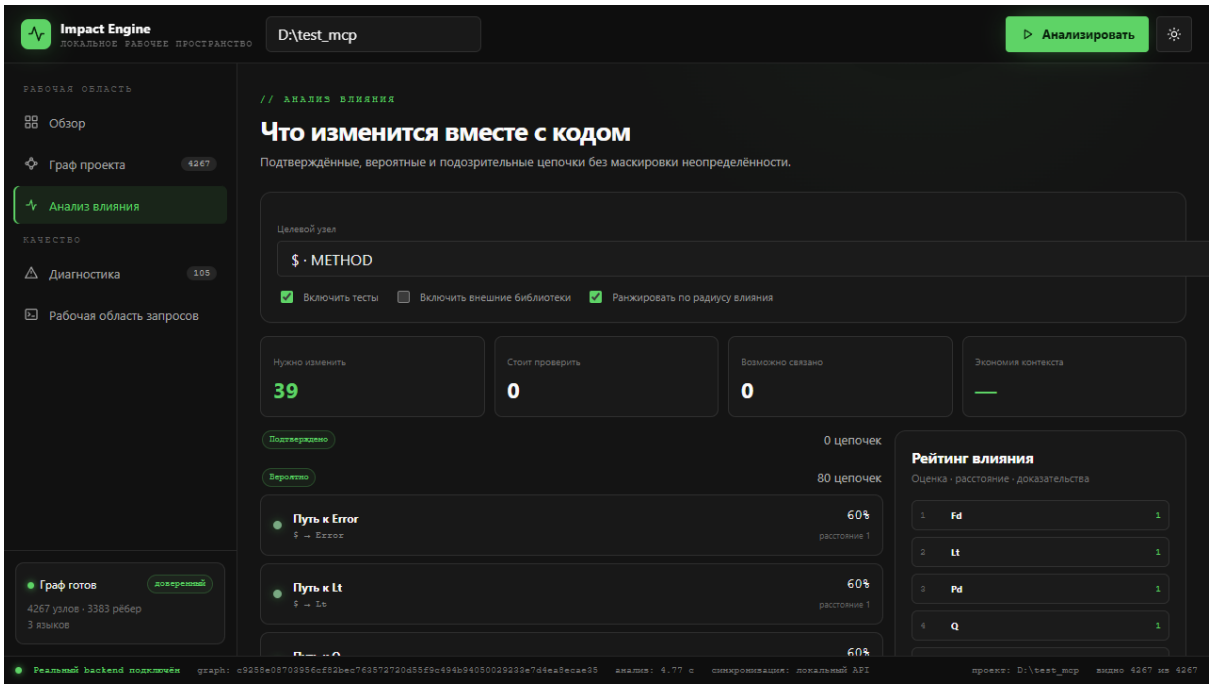


Рисунок 3 — impact-анализ CodeSlicer.

Экран показывает практический смысл интерфейса: вместо обычного списка файлов пользователь видит связанные элементы и может перейти к анализу последствий изменения.

5. Вывод

В лабораторной работе рассмотрена реальная клиентская часть CodeSlicer. Требования к использованию HTML, CSS и JavaScript реализованы внутри полноценного одностраничного интерфейса: HTML задаёт структуру, JavaScript хранит состояние, переключает маршруты, создаёт DOM-элементы, обрабатывает действия пользователя и управляет интерактивной SVG-картой.

Особенностью реализации является отсутствие статической имитации данных. После запуска bootstrap проверяет local API и получает

фактическое состояние проекта. При отсутствии анализа показывается onboarding-форма, а при ошибке соединения интерфейс сообщает о проблеме. Поэтому клиентская часть CodeSlicer является примером современного веб-приложения, где визуальный слой напрямую связан с серверной логикой и состоянием анализируемого проекта.

Визуальная концепция использует лаконичный подход современных AI-IDE, в том числе Kodik IDE, но функционально интерфейс подчинён собственной задаче CodeSlicer — показывать структуру проекта и последствия изменений.